# AI-Powered SEO Content Planner Using Multi-Agent LLMs

Complete Scope, Technical Architecture, and Problem-Solving Report

8th Semester Natural Language Processing (NLP)

## Course & Project Details

| | |
|---|---|
| **Course:** | Natural Language Processing (NLP) - 8th Semester CS |
| **Project Title:** | AI-Powered SEO Content Planner Using Multi-Agent LLMs |
| **Deployment:** | Streamlit Cloud + GitHub CI/CD Pipeline |
| **LLM Providers:** | Google Gemini, Groq (gpt-oss-20b), Cerebras (qwen-3.8-27b) |
| **Document Scope:** | Part 1: Full Scope, Architecture & Troubleshooting (Roman Urdu & English) |

**Guide Ka Maqsad (Purpose)**

Yeh document NLP 8th Semester project ki mukammal samajh, multi-agent LLM pipeline, codebase architecture, aur un tamam challenges ke practical solutions ko Roman Urdu aur English mein detail se explain karne ke liye tayyar kiya gaya hai. Is se project viva, evaluation aur presentation ke har sawal ka mukammal jawaz milta hai.

## Table of Contents (Fehrist)


- 1. Project Background & Scope: NLP mein traditional single-prompting ke nuqsaanat aur multi-agent orchestration ki zaroorat.
- 2. Core NLP Objectives: Factual grounding, schema compliance, strict token budgets, aur multilingual capabilities.
- 3. System Architecture & Workflow: End-to-end dataflow (Ingestion -> Web Search -> Dense Retrieval -> Agents -> Review -> Export).
- 4. The 7 Specialized AI Agents: Har agent ka role, system prompt rules, input schema, aur expected output.
- 5. RAG & Embedding Architecture: Multilingual E5 dense vectors aur resilient Fallback Hashing embedding model.
- 6. All 7 Technical Problems & Solutions: Groq 429 rate limit, Cerebras 80s latency, Hugging Face cloud block, Urdu script enforcement, etc.

- 7. Technology Stack & Why We Used Them: CrewAI, Pydantic v2, Streamlit, DuckDuckGo, FPDF2, UHarfbuzz ki selection reasons.
- 8. Academic Viva Preparation Guide: Evaluation metrics, hallucination mitigation techniques, aur future scope.

## 1. Project Background & Scope (Ibtidayi Khulasa)

Modern Natural Language Processing (NLP) mein jab hum kisi Large Language Model (jaise GPT-4, LLaMA, ya Gemini) ko ek single prompt dete hain ke 'Write a 1500-word SEO article with research and keywords', to model 3 baray masail ka shikaar hota hai: (1) Hallucination: model fake facts aur non-existent URLs invent kar leta hai; (2) Lack of Structure: content repetitive ho jata hai aur target word count ko control nahi kar pata; (3) Zero Auditability: koi aisi layer nahi hoti jo verify kare ke jo likha gaya hai wo kisi authentic source se match karta hai ya nahi.

Is 8th Semester NLP Project ka maqsad ek aisa Autonomous Multi-Agent AI System develop karna hai jo ek single LLM call par depend karne ke bajaye ek professional digital publication house ki tarah mukhtalif specialized agents mein kaam divide kare. Har agent ka apna aik makhsoos goal, strict Pydantic contract, aur review mechanism hota hai.

**Core Philosophy**

Separation of Concerns: Research karna alag agent ka kaam hai, Audience samajhna alag agent ka, SEO strategy banana alag agent ka, Outline banana alag ka, Article likhna alag ka, aur Fact-Checking Review karna alag ka!

## 2. Core NLP Objectives & Project Targets

- Strict Output Structure: Pydantic v2 data contracts enforce kiye gaye hain. Model free-text ke bajaye structured JSON schemas return karta hai jahan har field ka type validate hota hai.
- Retrieval-Augmented Generation (RAG): External authentic sources (PDF, DOCX, XLSX, Web) ko parse karke token-bounded chunks mein divide kiya jata hai aur cosine similarity se most relevant chunks select hotay hain.
- Hallucination Prevention & Peer Review: Ek independent Auditor Agent draft ki har line ko verify karta hai aur check karta hai ke kya yeh claims retrieved evidence se supported hain ya unsupported.
- Multilingual Generation Support: English ke ilawa authentic Urdu script (اردو رسم الخط) aur Roman Urdu mein complete research aur writing ki support.
- Production Cloud Readiness: Streamlit Community Cloud par continuous deployment, secret management, multi-provider fallbacks, aur offline-resilient zero-download embeddings.

## 3. System Architecture & Workflow (Pura System Kaise Kaam Karta Hai?)

System ka workflow 5 distinct sequential aur concurrent stages par mushtamil hai:

- Stage 1 - Ingestion & Candidate Discovery: User topic aur instructions provide karta hai. System user ke uploaded documents (PDF, Excel, PPTX, TXT) read karta hai aur DuckDuckGo Search API se live web references fetch karta hai.
- Stage 2 - Source Selection & Scraping: Source Selection Agent candidate URLs mein se top relevant sources filter karta hai. Web scraper (BeautifulSoup) un pages se boilerplates, ads, aur navbars remove karke clean text nikalta hai.
- Stage 3 - Token-Bounded Chunking & Dense Retrieval: Scraped text ko 240-word overlapping chunks mein divide kiya jata hai. SentenceTransformer (ya Fallback Tokenizer) se query aur chunks ke dense vectors bante hain aur cosine similarity se Top-5 evidence chunks select hotay hain.
- Stage 4 - Multi-Agent Drafting Ensemble (Parallelized): Audience Persona Agent aur SEO Strategist Agent research findings ke sath mil kar comprehensive Content Outline taiyar karte hain. Phir Writer Agent aur Unified Reviewer concurrently run hotay hain.
- Stage 5 - Quality Audit & Multi-Format Export: Auditor Agent claims ko check karke final status assign karta hai ('automated_review_passed' ya 'review_issues_found'). Result ko Markdown, JSON, aur Typeset PDF mein export kiya jata hai.

## 4. The 7 Specialized AI Agents & Their Roles

### >> Agent 1: Source Selection Agent (SourceSelector)

Role: External research librarian. Yeh agent web search se aane wale 5-10 URLs ke snippets scan karta hai aur un mein se sirf authentic, trustworthy aur relevant sources (2 to 3) ko content generation ke liye select karta hai. Commercial ads ya irrelevant links ko reject kar deta hai.

### >> Agent 2: Topic Researcher Agent (ResearchAgent)

Role: Factual analyst. Yeh agent retrieved evidence chunks ko analyze karta hai. Iska kaam sirf facts, statistics, aur authentic claims ko extract karna hai. Agar koi claim chunks mein mojood na ho, to yeh missing_information list mein daal deta hai.

### >> Agent 3: Audience Persona Agent (AudiencePersona)

Role: User empathy strategist. Yeh agent target readers (e.g. beginner university students, business owners) ke pain points, foundational prerequisite knowledge, aur reader assumptions ko map karta hai.

### >> Agent 4: SEO Strategist Agent (SEOStrategist)

Role: Search engine optimizer. Yeh agent search intent (informational, commercial, navigational) classify karta hai, Primary Keyword, Secondary Keywords, aur Long-tail Keywords generate karta hai jo modern Google Helpful Content update ke mutabiq hon.

### >> Agent 5: Content Structure Agent (StructureAgent)

Role: Chief editor / Architect. Yeh agent Research, SEO, aur Audience reports ko combine karke ek comprehensive H2/H3 outline taiyar karta hai, coverage notes assign karta hai, aur har section ke sath relevant chunk_ids bind karta hai.

### >> Agent 6: Content Writer Agent (WriterAgent)

Role: Lead author. Yeh agent outline aur evidence ko dekhte hue complete, engaging, human-readable article draft karta hai. Is par strict word count budget aur language script constraints apply hotay hain.

### >> Agent 7: Auditor & Reviewer Agent (ReviewerAgent)

Role: Quality & Fact-checking Auditor. Yeh agent writer ke draft ki har statement ko retrieved evidence chunks ke samnay rakh kar verify karta hai. Unsupported claims ko 'flag' karta hai aur correction requirements emit karta hai.

## 5. Dense Vector Embeddings & RAG Implementation

Retrieval-Augmented Generation (RAG) hamare system ka core factual pillar hai. RAG ke baghair LLM sirf training memory se likhta hai jahan outdated data aur hallucinations hoti hain. Hamara system 2-tier embedding model follow karta hai:

- Primary Model (Multilingual E5): Model: `intfloat/multilingual-e5-small`. Yeh 384-dimensional dense vectors generate karta hai jo 100+ languages (English, Urdu, Arabic) ko cross-lingual space mein align karta hai. 'passage: ' aur 'query: ' prefix use karta hai jo E5 architecture ke liye zaroori hain.
- Resilient Fallback Embedding Model: Agar Hugging Face unreachable ho ya cloud container offline ho, to system hamare custom zero-download `FallbackEmbeddingModel` par switch karta hai. Yeh MD5 feature hashing aur unigram/bigram term weighting ke zariye 384-dimensional unit vector banata hai. Yeh 0 MB disk space leta hai aur 0.001 second mein execute hota hai!

## 6. All 7 Technical Problems Faced & How We Solved Them

**Detailed Engineering Log**

Real-world NLP deployment mein aane wale tamam bottlenecks aur unke systematic solutions.

### >> Problem 1: Groq API Rate Limit (6,000 TPM) & 130s Pauses

- Issue: Groq ke free tier par 6,000 Tokens Per Minute (TPM) ki limit hai. Pehle code mein har stage par 3,000 completion tokens mangay ja rahe thay. 2-3 stages ke baad Groq HTTP 429 throw karta tha aur app ko 60-120 seconds wait karna parta tha.
- Solution: Humne `transport.py` mein `STAGE_TOKENS` dictionary introduce ki jo har stage ke role ke mutabiq completion tokens optimize karti hai (Intake: 400, SEO: 600, Research: 800, Structure: 1200). Is se total tokens 6,000 TPM ke safe bracket mein aa gaye aur pauses khatam ho gaye.

## >> Problem 2: Cerebras 80 Seconds Latency

- Issue: Cerebras API par `gpt-oss-120b` aik 120-Billion parameter reasoning model hai jo internal thinking tokens generate karta hai. 8 stages run hone mein 80 seconds lag rahay thay.
- Solution: Humne stage execution ko optimize kiya: (1) Writer aur Unified Review ko `asyncio.gather` se parallel execute kiya; (2) Cerebras ke liye high-speed `qwen-3.8-27b` model support add ki jo ~4 seconds mein response deta hai; (3) Stages ke darmiyan artificial delay `min_gap` ko 0.1s tak reduce kiya.

## >> Problem 3: Streamlit Cloud Hugging Face Block (OSError)

- Issue: Jab app Streamlit Community Cloud par deploy hui, to app crash ho gayi: `OSError: We couldn't connect to https://huggingface.co`. Cloud containers par Hugging Face hub ka 470MB model download Cloudflare ya firewalls ki wajah se block ho jata hai.
- Solution: Humne [engine.py] mein `FallbackTokenizer` aur `FallbackEmbeddingModel` likha. Agar Hugging Face offline ho ya download fail ho, to code crash hone ke bajaye seamlessly local hash-based embedding model par switch kar jata hai. Is se app 100% stable ho gayi.

## >> Problem 4: Urdu Selection par Roman Urdu Output Aana

- Issue: User jab 'Urdu' select karta tha, to model English alphabets mein Roman Urdu (e.g. 'Yeh article SEO ke baray mein hai') likh deta tha authentic Urdu script (اردو رسم الخط) ke bajaye.
- Solution: Humne Structure, SEO, aur Writer stages mein strict `CRITICAL LANGUAGE SCRIPT MANDATE` add kiya jo authentic Arabic/Persian letters enforce karta hai. Sath hi `[ـهـ-ـے]` regex validation lagai jo ensure karti hai ke draft authentic Urdu rasm-ul-khat mein hi ho.

## >> Problem 5: Target Word Count Overshooting (Target 450 -> 491 Words)

- Issue: User ne 450 words target select kiya lekin output 491 words ban rahi thi kyunki LLMs token prediction par kaam karte hain aur exact word count follow nahi karte.
- Solution: Humne `engine.py` mein dynamic `words_per_sec` token budgeting lagai aur healing step mein multilingual sentence boundary splitter `[.!?۔؟]` aur word-by-word terminal trimming add ki jo exact target (450 +/- 5 words) par article ko finish karti hai.

## >> Problem 6: PDF Export Unavailable on Cloud Container

- Issue: Streamlit Cloud par article generate hone ke baad 'Prepare PDF' click karne par error aata tha: `PDF export is temporarily unavailable`.
- Solution: Check karne par pata chala ke `requirements.txt` mein `fpdf2` aur `uharfbuzz` shamil nahi thay jis ki wajah se cloud server par `ModuleNotFoundError` aa raha tha. Humne dependencies add ki aur font fallback mechanism ko bulletproof banaya.

## >> Problem 7: Streamlit UI Sluggishness & State Reset Lag

- Issue: Fields switch karne par (English to Urdu ya Prompt to Detailed fields) Streamlit double re-render ki wajah se lag karta tha aur text truncate ho jata tha.
- Solution: Single-line `st.text_input` ko auto-expanding `st.text_area` mein convert kiya, session state ko widgets ke unique keys ke sath direct bind kiya, aur redundant re-sync code ko remove kiya.

## 7. Technology Stack & Why Each Tool Was Selected

- Python 3.10+: Asynchronous execution (`asyncio.gather`), typing, and rich NLP ecosystem ke liye industry standard.
- CrewAI + JSONTransport: Agent orchestration framework. Humne default LLM call ko override karke custom `JSONTransport` banaya taake har call deterministic aur pure JSON schema compliant ho.
- Pydantic v2: Runtime type checking aur data validation. Agar LLM kisi field ka naam galat kare ya schema violate kare, to Pydantic foran catch kar ke self-heal karta hai.
- Streamlit Cloud: Real-time interactive web UI jo browser mein seamless dashboard provide karta hai.
- DuckDuckGo Search: Zero-cost, keyless, privacy-friendly real-time web discovery tool jo authoritative sources dhoondta hai.
- FPDF2 + UHarfbuzz: Modern vector PDF generator with HarfBuzz shaping engine jo Urdu aur Arabic ligatures ko perfectly jorta hai.
- LLM Providers (Multi-Cloud): Google Gemini (1M TPM limit for heavy tasks), Groq (ultra-low inference latency), aur Cerebras (CS-3 wafer engine).

## 8. Academic Viva Preparation & Future Scope

NLP Course Evaluation mein examiners aam tor par yeh sawalat poochte hain:

- Q1: Aap ka system ChatGPT se behtar kaise hai? Answer: ChatGPT ek black-box single prompt hai jismein hallucinations aur unverified claims hotay hain. Hamara system multi-agent peer-review follow karta hai jahan har claim authentic chunk ID ke sath grounded hota hai aur auditor agent se verify hota hai.
- Q2: Multi-agent design ka NLP benefit kya hai? Answer: Cognitive load distribution. Ek LLM agar research, SEO, outline, writing aur auditing aik sath kare to context drift aur confusion hoti hai. Chotay specialized prompts model ki accuracy aur focus ko 3x barha dete hain.
- Q3: Hallucination kaise control hoti hai? Answer: (1) System prompt rules ungrounded claims ko forbid karte hain; (2) Pydantic schemas evidence quotes ko chunk text ke sath exact substring check karte hain; (3) Auditor agent claims ko source catalog se map karta hai.
- Future Work: Graph-RAG integration (knowledge graphs for deeper domain mapping), automated human-in-the-loop review interface, aur self-hosted open-weights models (Ollama/vLLM) ki local deployment.